UNIVERSIDAD AUSTRAL

INGENIERÍA
Departamento de Computación

APUNTES

Sistemas Distribuidos

Clase 04 – Consenso distribuido

Departamento de Computación

Ingeniería Informática – Comisión A

Pilar – Primer cuatrimestre 2026

Equipo docente

11 de agosto de 2026

1 Introducción

En la clase anterior estudiamos replicación y consistencia. Vimos que múltiples réplicas introducen divergencia debido a asincronía, fallas y concurrencia.

Esto nos lleva a un problema central:

> ¿Cómo imponemos un orden único de operaciones en un sistema distribuido?

La respuesta es el **consenso distribuido**.

> Consenso permite que múltiples nodos acuerden un único valor, lo que induce un orden global de operaciones.

—

2 Modelo del sistema

Trabajamos bajo el siguiente modelo:

- n procesos (nodos)
- comunicación por mensajes
- red asíncrona (sin límites de tiempo)
- fallas por crash (fail-stop)

No asumimos:

- relojes sincronizados
- tiempos máximos de entrega

—

3 Definición formal de consenso

Cada proceso p_i propone un valor v_i.

El algoritmo debe garantizar:

3.1 Agreement

> Si dos procesos deciden, deciden el mismo valor.

3.2 Validity

> El valor decidido fue propuesto por algún proceso.

3.3 Termination

> Todo proceso correcto eventualmente decide.

—

4 Imposibilidad: Teorema FLP

4.1 Enunciado

> En un sistema completamente asíncrono con al menos una posible falla, no existe algoritmo determinista que garantice consenso (terminación).

—

4.2 Intuición profunda

El problema no es Agreement ni Validity, sino **Termination**.

La red puede:

- retrasar mensajes arbitrariamente
- simular fallas (delay vs crash indistinguibles)

—

4.3 Estados de ejecución

Definimos:

- **Univalente**: el resultado está determinado (0 o 1)
- **Bivalente**: el resultado aún puede ser 0 o 1

El estado inicial es típicamente bivalente.

—

4.4 Idea de la prueba

1. Existe un estado inicial bivalente 2. Siempre existe una transición que mantiene bivalencia 3. El scheduler puede elegir esas transiciones indefinidamente

> Resultado: ejecución infinita sin decisión

—

4.5 Interpretación

No se puede garantizar consenso perfecto sin introducir supuestos adicionales (timeouts, sincronía parcial, etc.)

—

5 Consenso en la práctica

Los sistemas reales evaden FLP mediante:

- timeouts
- elecciones probabilísticas

- sincronía parcial

Esto permite progreso *en la práctica*, aunque no en el peor caso teórico.

—

6 Raft: consenso práctico

Raft es un algoritmo diseñado para ser comprensible y seguro.

—

6.1 Componentes

- Leader
- Followers
- Log replicado

—

6.2 Estructura temporal

El tiempo se divide en **términos** (terms).

- cada término tiene un líder
- los términos son monótonos

—

6.3 Elección de líder

1. follower expira timeout
2. se convierte en candidate
3. solicita votos
4. si obtiene mayoría $\rightarrow$ leader

—

6.4 Replicación de log

1. cliente envía request al leader
2. leader agrega entrada a su log
3. leader replica a followers
4. cuando mayoría confirma $\rightarrow$ commit

—

7 Invariantes de Raft

Estos garantizan **seguridad**.

—

7.1 Log Matching Property

Si dos logs tienen una entrada en el mismo índice y término, entonces todos los elementos anteriores son idénticos.

Razón:

- el leader solo replica desde prefijos consistentes
- followers rechazan inconsistencias

—

7.2 Leader Completeness

Si una entrada está comprometida, aparecerá en todos los líderes futuros.

Idea:

- commit requiere mayoría
- futuros líderes también requieren mayoría
- intersección $\Rightarrow$ preservación

—

8 Quorums

Un quorum es un subconjunto de nodos que puede tomar decisiones.

Para consenso:

$$|Q| > \frac{N}{2}$$

—

8.1 Propiedad clave

$$Q_1 \cap Q_2 \neq \emptyset$$

Demostración:

Supongamos dos quorums disjuntos:

$$|Q_1| + |Q_2| \leq N$$

Pero:

$$|Q_1| > N/2, \quad |Q_2| > N/2$$

$$|Q_1| + |Q_2| > N$$

Contradicción.

—

9 Paxos

Paxos es el algoritmo clásico de consenso.

—

9.1 Roles

- Proposer
- Acceptor
- Learner

—

9.2 Fases

Phase 1 (Prepare)

- proposer envía número n
- acceptors prometen no aceptar menor

Phase 2 (Accept)

- proposer propone valor
- acceptors aceptan si cumple promesa

—

10 Propiedad de seguridad de Paxos

> Si un valor es elegido, ningún otro valor distinto puede ser elegido.

—

10.1 Prueba (idea)

- elección requiere mayoría
- nuevas propuestas deben consultar mayoría
- intersección $\Rightarrow$ conocimiento previo

Por lo tanto:

no puede aparecer un valor distinto

—

11 Consenso y linearizabilidad

El consenso permite implementar:

- orden total de operaciones
- máquina de estados replicada

Consenso ⇒ orden total ⇒ linearizabilidad

—

12 Costo del consenso

- múltiples rondas de comunicación
- necesidad de mayoría
- latencia
- menor disponibilidad bajo partición

—

13 Modelo del curso

$$D = (P, F, G, S, T)$$

- P: estado consistente global
- F: fallas y asincronía
- G: consenso (agreement)
- S: Raft/Paxos
- T: latencia y disponibilidad

—

14 Conclusión

Consenso es el mecanismo que transforma un sistema distribuido en una máquina secuencial lógica.

Pero:

- no es gratis
- no siempre progresa
- requiere compromisos explícitos

El consenso no elimina la incertidumbre: la controla.

15 Síntesis razonada de la clase

Esta unidad debe leerse como parte de una cadena de decisiones de diseño y no como un catálogo de mecanismos. Para estudiar el tema con profundidad conviene reconstruir cuatro preguntas: *qué problema aparece*, *qué garantía se desea*, *qué mecanismo la aproxima* y *qué costo introduce*. En cada ejemplo debe identificarse además el modelo de falla implícito.

Una respuesta técnicamente madura no termina al nombrar un algoritmo o patrón: declara sus supuestos, la propiedad que preserva y el escenario en el que deja de ser suficiente.

16 Bibliografía guiada y ruta de profundización

Para profundizar esta clase se recomienda: Ongaro y Ousterhout (Raft); Lamport (Paxos); Fischer, Lynch y Paterson (FLP); Kleppmann, cap. 9.

La lectura debe hacerse con preguntas concretas: (1) ¿qué modelo de sistema asume el autor?, (2) ¿qué propiedad demuestra o persigue?, (3) ¿qué falla o anomalía motiva el mecanismo?, y (4) ¿qué trade-off queda abierto?

17 Conexión con el Trabajo Final Integrador

El grupo debe señalar qué parte de su sistema utiliza los conceptos de esta unidad, justificar por qué son necesarios y documentar una alternativa descartada. Cuando el contenido todavía no corresponda a la etapa vigente, debe registrarse como hipótesis o decisión pendiente.

18 Autoevaluación conceptual

1. ¿Cuál es el problema distribuido concreto que motiva esta clase?
2. ¿Qué propiedad de seguridad y qué propiedad de progreso están en juego?
3. ¿Qué supuesto de red, tiempo o falla resulta decisivo?
4. ¿Qué trade-off aceptaría en un sistema real y cuál no aceptaría en un dominio crítico?
5. ¿Cómo observaría experimentalmente que la solución se comporta como espera?

19 Profundización autocontenida v9

19.1 Problema, límite e invariante

El núcleo de esta clase es consenso, FLP, mayorías, Raft, Paxos, seguridad, progreso y máquina de estados replicada. Para estudiarlo de manera autónoma conviene separar tres planos. Primero, el *problema observable*: qué comportamiento incorrecto puede ver un cliente o un operador. Segundo, el *límite del modelo*: qué información, sincronía o coordinación no está disponible. Tercero, el *invariante*: qué propiedad no debe violarse aunque el sistema pierda progreso temporalmente.

El modelo de fallas relevante incluye crash, demora arbitraria, elección dividida y pérdida temporal de mayoría. Una solución debe describirse como protocolo y no solo como nombre de tecnología: actores, estados, mensajes, condición de éxito, condición de aborto y estado visible después de una interrupción. La evidencia esperada es términos, votos, logs, commitIndex y un invariante de seguridad.

19.2 Método de análisis

Ante un caso nuevo, siga esta secuencia:

1. Identifique actores, dato o recurso compartido y frontera de confianza.
2. Escriba una ejecución nominal y una ejecución adversarial.
3. Distinga seguridad (lo malo no ocurre) de progreso (algo bueno finalmente ocurre).
4. Declare qué supuesto permite avanzar: mayoría, deadline, sincronía parcial, operación idempotente, almacenamiento durable u otro.
5. Compare una alternativa más simple y una más fuerte; registre qué métrica o experimento haría cambiar la decisión.

19.3 Caso transferible

Considere una plataforma de reservas que recibe operaciones duplicadas, pierde conectividad entre regiones y debe sostener un camino crítico sin ocultar el estado real al usuario. Aplique el mecanismo de esta clase a una única decisión concreta. Una respuesta madura no intenta resolver todo: delimita la operación, formula la garantía y explica la degradación esperada cuando el supuesto central deja de cumplirse.

19.4 Errores de razonamiento frecuentes

- confundir una propiedad con el producto que suele implementarla;
- describir el camino feliz sin recorrer una falla intermedia;
- afirmar “exactly-once”, “alta disponibilidad” o “consistencia” sin definir alcance observable;
- medir solo throughput aunque la hipótesis trate sobre semántica o recuperación;
- presentar la complejidad añadida como beneficio sin compararla con la alternativa más simple.

20 Lectura guiada

Ongaro y Ousterhout, Raft; Lamport, Paxos Made Simple; material MIT 6.5840 sobre consenso.

Durante la lectura, registre: modelo de sistema, propiedad perseguida, contraejemplo que motiva el mecanismo, costo principal y una condición bajo la cual no lo elegiría. Estas cinco anotaciones convierten la bibliografía en una herramienta de diseño y no en una lista para memorizar.

21 Aplicación al Trabajo Final Integrador

El equipo debe producir un ADR breve con la decisión asociada a esta clase. Debe contener: contexto, dos alternativas, supuesto decisivo, garantía elegida, trade-off, evidencia pendiente y fecha de revisión. La evidencia mínima esperada para esta unidad es: términos, votos, logs, commitIndex y un invariante de seguridad.

22 Autoevaluación avanzada

1. ¿Puedo explicar la clase sin nombrar primero una tecnología?
2. ¿Puedo construir una ejecución que viole la garantía si el mecanismo se configura mal?
3. ¿Puedo distinguir seguridad, progreso y experiencia del usuario?
4. ¿Puedo proponer una medición que valide la propiedad, no solo el rendimiento?
5. ¿Puedo defender cuándo una solución más simple sería suficiente?